

NOTES

List Object Basics



List Object Basics

Introduction

In Python, a **list** is a built-in data structure that is used to store multiple items in a single variable. Lists are mutable, ordered, and can contain elements of different data types.

1. Declaration of List

To declare a list in Python, you use square brackets `[]`. A list can contain multiple elements, including numbers, strings, or even other lists. Below are different ways to declare a list in Python:

Properties of List

You can declare an empty list by simply using empty square brackets.

- **Ordered:** Maintains the order of elements.
- **Mutable:** Allows modification of elements.
- **Heterogeneous:** Can contain different data types.
- **Indexing:** Supports accessing elements via indices.
- **Slicing:** Enables extraction of sublists.
- **Duplicates:** Permits multiple occurrences of the same element.
- **Built-in Methods:** Offers various methods for manipulation.

```
Python
my_list = []
print(my_list)  # Output: []
```

Declaration of List with Elements

You can declare a list with elements by placing them inside the square brackets, separated by commas.

```

Python
# List with integers
my_list = [1, 2, 3, 4, 5]

# List with strings
fruits = ["apple", "banana", "cherry"]

# List with mixed data types
mixed_list = [1, "apple", True, 3.14]

print(my_list)      # Output: [1, 2, 3, 4, 5]
print(fruits)       # Output: ['apple', 'banana', 'cherry']
print(mixed_list)   # Output: [1, 'apple', True, 3.14]

```

List of Lists (Nested Lists)

You can also declare a list inside another list, known as a nested list.

```

Python
nested_list = [[1, 2, 3], ["a", "b", "c"], [True, False]]
print(nested_list)

# Output: [[1, 2, 3], ['a', 'b', 'c'], [True, False]]

```

Using the *list()* Constructor

You can create a list by converting other iterables (like strings, tuples, or ranges) using the `list()` constructor.

```

Python
# Converting a string to a list
str_list = list("hello")
print(str_list)  # Output: ['h', 'e', 'l', 'l', 'o']

# Converting a tuple to a list
tuple_list = list((1, 2, 3))
print(tuple_list)  # Output: [1, 2, 3]

# Creating a list from a range
range_list = list(range(5))
print(range_list)  # Output: [0, 1, 2, 3, 4]

```

2. List Operations

Creating a List

A list is created using square brackets `[]`, and it can store multiple data types.

```
Python  
my_list = [1, 2, 3, "apple", True]
```

Accessing Elements

You can access elements using their index. Lists are zero-indexed.

```
Python  
print(my_list[0]) # Output: 1  
print(my_list[-1]) # Output: True (negative indexing starts from the  
end)
```

Modifying Elements

Lists are mutable, so you can change their elements.

```
Python  
my_list[0] = 100  
print(my_list) # Output: [100, 2, 3, 'apple', True]
```

Appending and Extending Lists

- `append(item)`: Adds an item to the end of the list.
- `extend(iterable)`: Adds all elements from an iterable (like another list).

```
Python  
my_list.append("orange") # Output: [100, 2, 3, 'apple', True,  
'orange']  
my_list.extend([4, 5]) # Output: [100, 2, 3, 'apple', True,  
'orange', 4, 5]
```

Inserting Elements

Use `insert(index, item)` to insert an item at a specific index.

```
Python
my_list.insert(1, "banana") # Output: [100, 'banana', 2, 3, 'apple',
True, 'orange', 4, 5]
```

Removing Elements

- `remove(item)`: Removes the first occurrence of the item.
- `pop(index)`: Removes and returns the item at the given index. If no index is provided, it removes the last item.

```
Python
my_list.remove("apple") # Output: [100, 'banana', 2, 3, True,
'orange', 4, 5]
my_list.pop(0)          # Output: [100] is removed, list becomes
['banana', 2, 3, True, 'orange', 4, 5]
```

List Length

Use `len(list)` to get the number of elements in a list.

```
Python
print(len(my_list)) # Output: 7
```

List Slicing

You can slice lists using the syntax `list[start:stop:step]`.

```
Python
sliced_list = my_list[1:4]
# Output: ['banana', 2, 3]
```

List Sorting

- `sort()`: Sorts the list in place.
- `sorted(list)`: Returns a new sorted list.

```
Python
numbers = [5, 1, 4, 2]
numbers.sort() # Output: [1, 2, 4, 5]
```

List Comprehension

A concise way to create lists.

```
Python
squares = [x2 for x in range(5)]
# Output: [0, 1, 4, 9, 16]
```

List Membership

Use `in` to check if an item exists in a list.

```
Python
print(2 in my_list) # Output: True
```

Copying a List

- `list.copy()` or `list[:]`: Creates a shallow copy.

```
Python
copied_list = my_list.copy()
# Creates a copy of the list
```

Clearing a List

- `clear()`: Removes all elements from the list.

```
Python
my_list.clear() # Output: []
```

3. List Comprehensions

List comprehension in Python is a concise and efficient way to create lists by performing operations on existing iterables (like lists, strings, or ranges). It allows you to write a for-loop inside square brackets to generate a list in a single line, making your code more readable and compact.

```
Python  
new_list = [expression for item in iterable if condition]
```

Creating a List of Squares

This example, shows how to create a list of squares using list comprehension:

```
Python  
squares = [x**2 for x in range(5)]  
print(squares) # Output: [0, 1, 4, 9, 16]
```

Filtering with List Comprehension

You can also add a condition (using an 'if' statement) to filter elements.

```
Python  
even_numbers = [x for x in range(10) if x % 2 == 0]  
print(even_numbers) # Output: [0, 2, 4, 6, 8]
```

List Comprehension with Strings

List comprehension works with strings too. For 3., creating a list of characters from a string:

```
Python  
chars = [char for char in "hello"]  
print(chars) # Output: ['h', 'e', 'l', 'l', 'o']
```

Nested Loops in List Comprehension

You can nest loops inside list comprehension to create more complex lists, such as a list of pairs.

```
Python
pairs = [(x, y) for x in range(3) for y in range(2)]
print(pairs)
# Output: [(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

List Comprehension with If-Else

You can also include an `if-else` condition in the expression part of list comprehension:

```
Python
numbers = [x if x % 2 == 0 else -x for x in range(5)]
print(numbers) # Output: [0, -1, 2, -3, 4]
```

Benefits of List Comprehension:

- **Conciseness:** Reduces multiple lines of code into one line.
- **Readability:** Makes code easier to understand at a glance (once you are familiar with the syntax).
- **Performance:** Often faster than traditional loops because of the underlying optimizations.

Equivalent Code Using Loops:

To compare, here's how you would create a list of squares using a traditional `for` loop:

```
Python
squares = []
for x in range(5):
    squares.append(x**2)

print(squares) # Output: [0, 1, 4, 9, 16]
```


Now let's do the same thing with list comprehension

Python

```
squares = [x**2 for x in range(5)]
```



THANK YOU

